

レーダーモジュール評価キット

TITAN T14RE

Python サンプルコード

スタートガイド

エスタカヤ電子工業株式会社

もくじ

1. 概要	2
2. ソフトウェア構成	2
3. 動作確認	3
3.1. ファイル構成	3
3.2. コードの実行	4
4. プログラム解説	5
4.1. DLL の参照設定	5
4.2. 通信用インスタンスの作成	5
4.3. Cfg ファイルの送信	5
4.4. データの取得	6
4.5. データの取得停止	6
5. 注意事項	7

1. 概要

TITAN T14RE レーダーモジュール(以下、本機)からデータを取得する Python サンプルコードです。

Windows では Class Library for T14RE(T14re.dll)を使用して簡単に取り込みを行うことができますが、本サンプルコードでは専用の DLL を使用せず、pyserial と OpenCV を使用して実装しており、Linux でも動作を行うことができます。DLL を使用する方法は C#サンプルプロジェクト T14reSampleSimple をご参照ください。

2. ソフトウェア構成

Windows の動作確認は下記の環境で行っています。

項目	説明
PC	下記のスペック以上 CPU: Intel Core i5 RAM: 8GB
OS	Windows10 Microsoft .NET Framework 4.7.2 以降 Microsoft Visual C++ 2015 Redistributable(x86)
実行環境	Python 3.9.5 pyserial 3.5 numpy 1.22.3 opencv-python 4.5.5.64

Linux の動作確認は下記の環境で行っています。

項目	説明
PC	ASUS 社製シングルボードコンピュータ tinker board 2s CPU: Rockchip RK3399 RAM: 2GB
OS	Debian GNU/Linux 10
実行環境	Python 3.7.3 pyserial 3.5 numpy 1.21.5 opencv-python 4.5.5.64

3. 動作確認

ここではサンプルコードの動作確認の方法を説明します。

3.1. ファイル構成

Windows 用であれば、“Windows”、Linux 用であれば“Linux”フォルダにあるファイルを使用します。

実行には下記のファイルが必要になりますので、これらが同じフォルダに存在していることを確認してください。

ファイル	説明
T14reSample.py	サンプルコードです
T14RE_3D_Short.cfg	RAW データ取り込み用の Cfg ファイルです

3.2. 準備

実行のためには pyserial、opencv-python のインストールが必要です。

3.3. 接続情報の取得と設定

Cfg ファイルを送信するための MMIC コマンドインターフェースと、データを取得するためのデータインターフェースのキャプチャ ID を取得する必要があります。

Windows の場合、T14reTool を使用して取得することができます。PC に本機を接続して、コマンドプロンプトで“T14reTool -deviceInfo”を実行すると、CSV 形式の応答が得られます。CommandIfPortName が MMIC コマンドインターフェースで、DataIfCaptureDeviceIndex がデータインターフェースのキャプチャ ID です。下記の例では“COM19”と“1”です。

```
C:\T14re>t14retool -deviceInfo
DeviceIndex,DeviceName,DataIfFriendlyName,DataIfCaptureDeviceIndex,CommandIfPortName,ControlIfPortName,UsbSpeed
0,DEBUG_2D,ST219-0033(Data):DEBUG_2D,1,COM19,COM18,3
```

Linux の場合は MMIC コマンドインターフェースを“ls -l /dev/serial/by-id”で、データインターフェースのキャプチャ ID を“ls -l /dev/v4l/by-id”で取得することができます。

PC に本機を接続して、ターミナルで“ls -l /dev/serial/by-id”を実行すると、シリアルインターフェースの一覧が表示されます。名前の後に-if02 についているものが MMIC コマンドインターフェースです。下記の例では“/dev/ttyACM0”です。

```
linaro@linaro-alip:~/Documents$ ls -l /dev/serial/by-id
total 0
lrwxrwxrwx 1 root root 13 Mar 29 02:30 usb-S-TAKAYA_ELECTRONICS_INDUSTY_CO._LTD._ST219-0033_DEBUG_2R5D_001-if02 -> ../../ttyACM0
lrwxrwxrwx 1 root root 13 Mar 29 02:30 usb-S-TAKAYA_ELECTRONICS_INDUSTY_CO._LTD._ST219-0033_DEBUG_2R5D_001-if04 -> ../../ttyACM1
```

続いて“ls -l /dev/v4l/by-id”を実行すると、ビデオデバイスの一覧が表示されます。名前の後に-video-index0 となっているものがデータインターフェースで、デバイスの最後の数字がキャプチャ ID です。下記の例では“5”です。

```
linaro@linaro-alip:~/Documents$ ls -l /dev/v4l/by-id
total 0
lrwxrwxrwx 1 root root 12 Mar 29 02:30 usb-S-TAKAYA_ELECTRONICS_INDUSTY_CO._LTD._ST219-0033_DEBUG_2R5D_001-video-index0 -> ../../video5
lrwxrwxrwx 1 root root 12 Mar 29 02:30 usb-S-TAKAYA_ELECTRONICS_INDUSTY_CO._LTD._ST219-0033_DEBUG_2R5D_001-video-index1 -> ../../video6
```

次に取得した MMIC コマンドインターフェースとデータインターフェースのキャプチャ ID をコードに設定します。
下記は Windows の場合の例です。

```
# 接続する T14RE のキャプチャ ID の設定
CaptureDeviceIndex = 1
# 接続する T14RE の MMIC コマンドインターフェースの設定
MmicCommandPort = 'COM19'
```

下記は Linux の場合の例です。

```
# 接続する T14RE のキャプチャ ID の設定
CaptureDeviceIndex = 5
# 接続する T14RE の MMIC コマンドインターフェースの設定
MmicCommandPort = '/dev/ttyACM0'
```

3.4. コードの実行

サンプルコードを実行すると Cfg ファイルが送信されデータの取り込みが開始されます。データ取り込み後、CSV 形式で出力します。

4. プログラム解説

ここではサンプルコードの説明を行います。Cfg ファイルの送信手順、取得データをデコードするためのフォーマットはソフトウェア説明書に記載しておりますので、最初にこちらをご参照ください。

4.1. Cfg ファイルの送信

Cfg ファイルの送信は MMIC コマンドインターフェースを使用してシリアル通信で行います。シリアル通信は pyserial を使用しています。

Cfg ファイルを読み込み、先頭から 1 行毎に、送信し応答を待つという処理を、最終行まで行います。

4.2. VideoCapture の設定

データの取得は OpenCV の VideoCapture を使用して行います。バックエンドは Windows が Microsoft Media Foundation、Linux が V4L2 です。

下記は Windows の場合の設定です。

```
capture = cv2.VideoCapture(CaptureDeviceIndex, cv2.CAP_MSMF)
```

下記は Linux の場合の設定です。

```
capture = cv2.VideoCapture(CaptureDeviceIndex, cv2.CAP_V4L2)
```

本機のデータは YUY2 ですが、画像ではなく RAW データまたは FFT データです。従って、RGB 変換せずにデータを受取るように設定をする必要があります。

```
capture.set(cv2.CAP_PROP_CONVERT_RGB, 0)
capture.set(cv2.CAP_PROP_FORMAT, -1)
```

続いて取り込むデータサイズの指定を行います。サイズの指定は 2 次元データフォーマットとして幅と高さを指定します。

```
capture.set(cv2.CAP_PROP_FRAME_WIDTH, Sample * 2 * Rx)
capture.set(cv2.CAP_PROP_FRAME_HEIGHT, Tx * Chirpset * Frame)
```

4.3. データの取得

データを取得するには、VideoCapture::read()を呼び出します。特定の実行環境では本機から最初のデータが取得できるまでに、何回か All-0 のデータを受信することがありますので、必要に応じてスキップしてください。

```
ret, data = capture.read()
```

4.4. データのデコード

取得したデータはデータフォーマットに合わせてデコードを行います。

Windows の場合は 1 次元データフォーマットです。サンプルコードではフレーム、チャープセット、送信アンテナ、受信アンテナ、サンプル位置をループさせて、ループの中でインデックスを増加させながら順にデコードしています。

```
index = 0
(ループ)
    valueI = data[0][index + 0] + data[0][index + 1] * 256
    if (valueI >= 0x8000):
        valueI -= 0x10000

    valueQ = data[0][index + 2] + data[0][index + 3] * 256
    if (valueQ >= 0x8000):
        valueQ -= 0x10000

    index += 4
```

Linux の場合は 2 次元データフォーマットです。サンプルコードではフレーム、チャープセット、送信アンテナ、受信アンテナ、サンプル位置をループさせて、それを 2 次元のデータの水平位置と垂直位置に変換してデコードしています。

```
(ループ)
    line = (fr * (Chirpset * Tx)) + (cs * Tx) + tt
    width = (rr * Sample * 2) + (ss * 2)

    valueI = data[line][width][0] + data[line][width][1] * 256
    if (valueI >= 0x8000):
        valueI -= 0x10000

    valueQ = data[line][width + 1][0] + data[line][width + 1][1] * 256
    if (valueQ >= 0x8000):
        valueQ -= 0x10000

    index += 4
```

4.5. データの取得停止

データの取得を停止するには、VideoCapture::release()を使用します。

```
capture.release()
```

5. 注意事項

- 本製品またはソフトウェアの使用により生じたいかなる損害にも当社は責任を負いません
- ソフトウェアの仕様はお断りなく変更する場合があります
- Microsoft および Windows は、米国 Microsoft Corporation の、米国およびその他の国における登録商標または商標です
- Linux は、米国およびその他の国における Linus Torvalds の登録商標です
- OpenCV は、米国およびその他の国における Intel Corporation の商標です
- その他、記載されている会社名、商品名は各社の登録商標または商標です
- ご使用の際は本製品に添付されたマニュアルをお読みになり、正しく安全にご使用ください

変更履歴

日付	Rev	担当	内容
2022/6/8	1.0	田口誠	初回作成
2022/6/14	1.1	田口誠	表現統一のための変更
2022/8/25	1.2	田口誠	誤記訂正
			以上